

Module - 1

Algorithm and Flowchart, Elements of Python: Keywords, Identifiers, Variables, Data Types, Features. **Operators and Expression:** Assignment Statements, Numeric Expressions, Order of Evaluation, Operator Precedence, Type Conversions, Input Output Statement, Comments in Python.

Basics of Python

Introduction to Python

Python is one of the most popular programming languages. It's simple to use, packed with features, and supported by a wide range of libraries and frameworks. Its clean syntax makes it beginner-friendly.

- A high-level language, used in web development, data science, automation, AI and more.
- Known for its readability, which means code is easier to write, understand and maintain.
- Backed by library support, so we don't have to build everything from scratch, there's probably a library that already does what we need.

Why to Learn Python?

- Requires fewer lines of code compared to other programming languages like Java.
- Provides Libraries / Frameworks like Django, Flask and many more for Web Development, and Pandas, TensorFlow, Scikit-learn and many more for, AI/ML, Data Science and Data Analysis
- Cross-platform, works on Windows, Mac and Linux without major changes.
- Used by top tech companies like Google, Netflix and NASA.
- Many Python coding job opportunities in Software Development, Data Science and AI/ML.

Features of Python

- **Free and Open Source** – Python can be downloaded freely, and its source code is open to the public.
- **Easy to Code** – Simple and beginner-friendly compared to other languages like C, C++ or Java.
- **Easy to Read** – Syntax is clean and uses indentation instead of brackets.
- **Cross-Platform** – runs on multiple operating systems
- **Object-Oriented** – Supports classes, objects, and OOP concepts.
- **Dynamically Typed** – no need to declare variable types explicitly.

-
- **GUI Support** – Libraries like Tkinter, PyQt allow making desktop applications.
- **High-Level Language** – No need to manage memory or system details manually.
- **Large Standard Library** - Python has a large standard library that provides a rich set of modules and functions so you do not have to write your own code for every single thing. There are many libraries present in Python such as regular expressions, unit-testing, web browsers, etc.
- **Large Community** – Huge online support through forums like StackOverflow.
- **Easy Debugging** – Clear error messages help find and fix mistakes quickly.
- **Portable** – The Same code can run on Windows, Linux, or Mac without changes.
- **Integrated** – Can be combined with other languages like C/C++.

What is an Algorithm?

The word Algorithm means “a process or set of rules to be followed in calculations or other problem-solving operations”. Therefore Algorithm refers to a set of rules/instructions that step-by-step define how a work is to be executed in order to get the expected results. Let's take a look at an example for a better understanding.

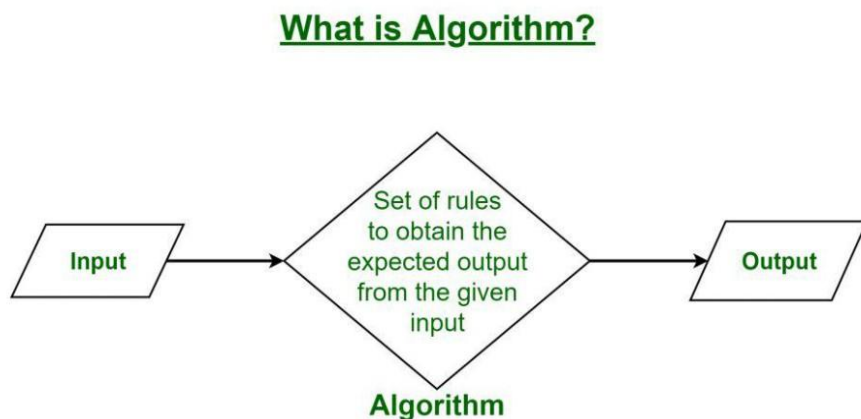


Figure 1.1: Basic Algorithm Model

What is a Flowchart?

A Flowchart is a graphical representation of an algorithm. Programmers often use it as a program-planning tool to solve a problem. It makes use of symbols that are connected among them to indicate the flow of information and processing. The process of drawing a flowchart for an algorithm is known as “flowcharting”.

Advantages of flowchart:

- Flowchart is an excellent way of communicating the logic of a program.

- Easy and efficient to analyze problem using flowchart.
- During program development cycle, the flowchart plays the role of a blueprint, which makes program development process easier.
- The flowchart makes program easier.
- It is easy to convert the flowchart into any programming language code.

Example: Draw a flowchart to input two numbers from the user and display the largest of two numbers.

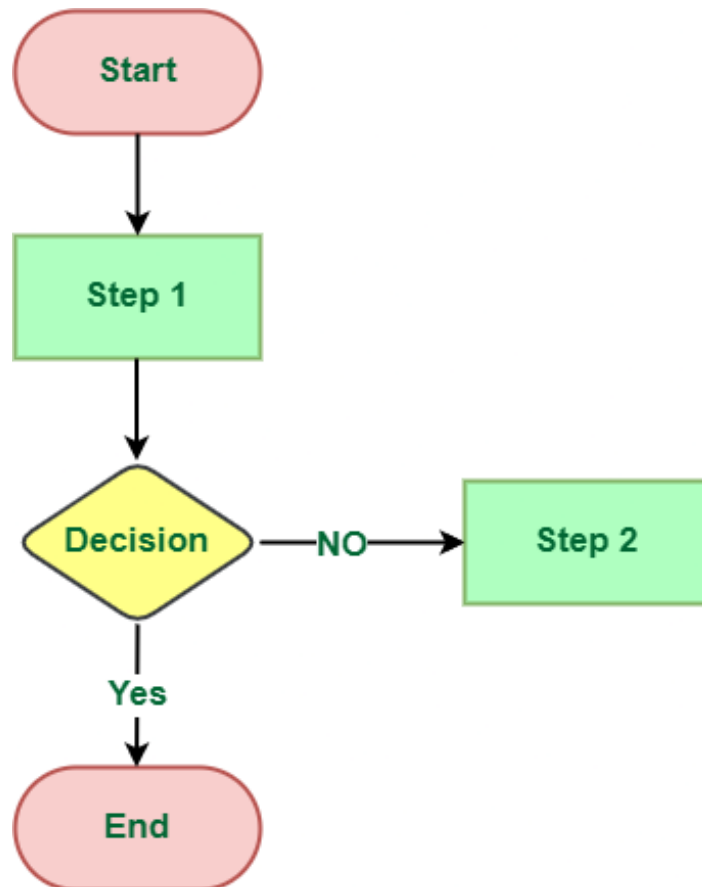


Figure 1.2: Flowchart with decision making

Write an algorithm to check whether a given number is even or odd.

Step 1: Start
 Step 2: Input a number n
 Step 3: Compute Remainder $n \% 2$
 Step 4: If remainder == 0 then
 Print "even number"
 Else
 Print "odd number"
 Step 5: Stop

Flowchart:

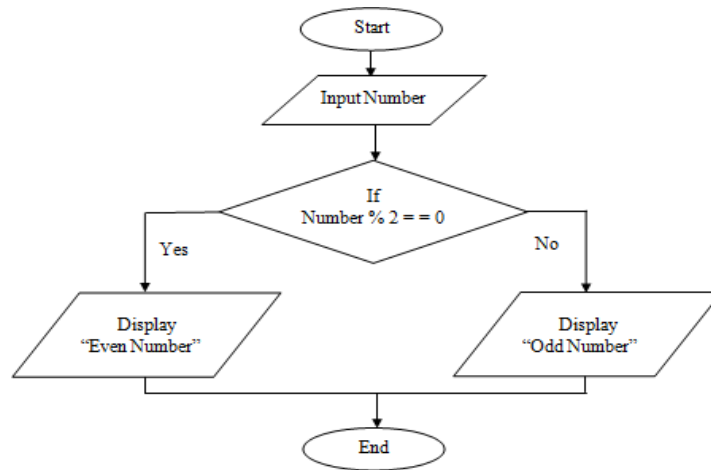


Figure 1.3: Flowchart for even or odd

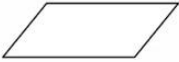
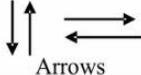
Name	Symbol	Purpose
Terminal	 oval	Start /stop/begin/end
Input / output	 Parallelogram	Input/output of data
Process	 Rectangle	Any processing to be performed can be represented
Decision box	 Diamond	Decision operations that determine which of the alternative paths to be followed
Connector	 Circle	Used to connect different parts of flowchart
Flow	 Arrows	Joins 2 symbols and also represents flow of execution
Pre defined process	 Double sided rectangle	Modules (or)subroutines specified else where
For loop symbol	 Hexagon	Shows initialization, condition and incrimination of loop variable.
Document	 Printout	Shows the data that is ready for printout

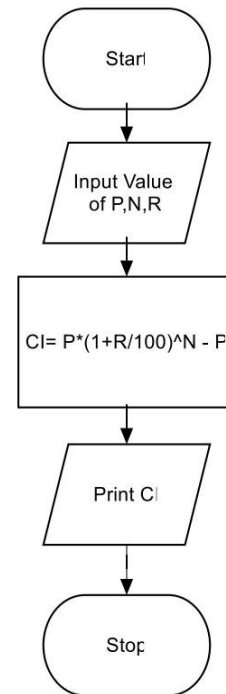
Table 1.1: Flowchart Symbols

Algorithm & Flowchart to find Compound Interest

P : Principle Amount
N : Time in Years
R : % Annual Rate of Interest
CI : Compound Interest

Algorithm

- Step-1 Start
- Step-2 Input value of P, N, R C
- Step-3 $CI = P(1+R/100)^N - P$
- Step-4 Display CI
- Step-6 Stop



..

Figure 1.4: Algorithm & Flowchart to find Compound Interest

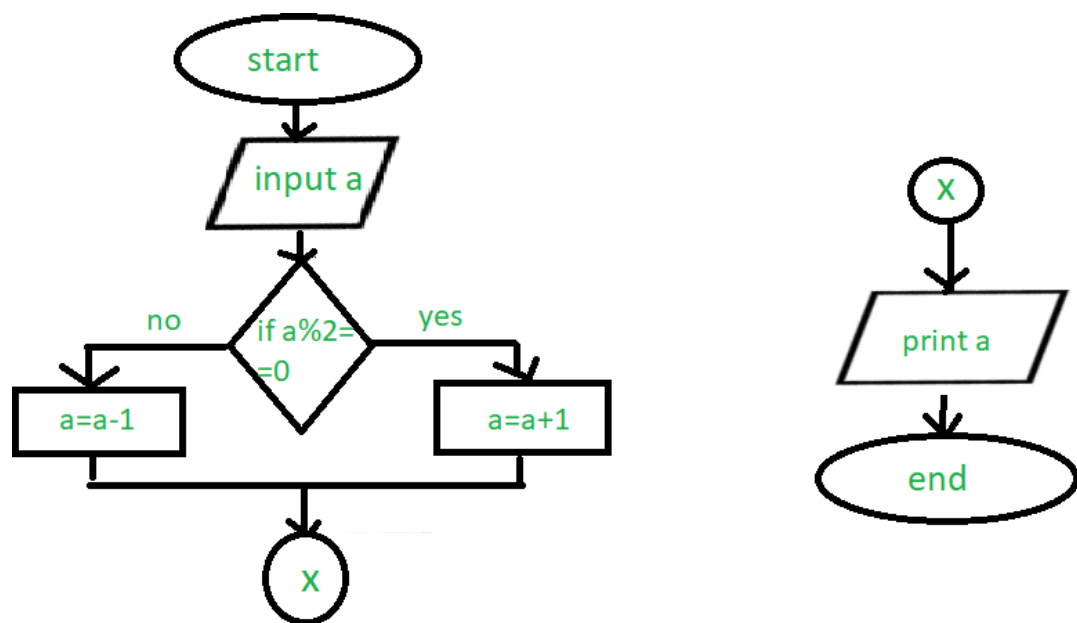


Figure 1.5: Flowchart

Python Keyword

Keywords in Python are special reserved words that are part of the language itself. They define the rules and structure of Python programs, which means you cannot use them as names for your variables, functions, classes, or any other identifiers.

Keywords in Python

- Predefined and reserved words with special meanings.
- Used to define the syntax and structure of Python code.
- Cannot be used as identifiers, variables, or function names.
- Written in lowercase, except True and False.
- Python 3.11 has 35 keywords.

Examples of Keyword

false	class	finally	is	return
none	continue	for	lambda	try
true	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	
assert	else	import	pass	
break	except	in	raise	

Table 1.2: Python Keywords

Python Identifier

- User-defined names for variables, functions, classes, modules, etc.
- Can include letters, digits, and underscores (_).
- Case-sensitive → num, Num, and NUM are different identifiers.
- Python provides str.isidentifier() to check if a string is a valid identifier.

Rules for Naming Python Identifiers

- It cannot be a reserved python keyword.
- It should not contain white space.
- It can be a combination of A-Z, a-z, 0-9, or underscore.
- It should start with an alphabet character or an underscore (_).
- It should not contain any special character other than an underscore (_).

Examples of Python Identifiers

Valid identifiers:

- var1
- _var1
- _1_var
- var_1

Invalid identifiers:

- !var1
- 1var
- 1_var
- var#1
- var 1

Python Variable

In Python, variables are used to store data that can be referenced and manipulated during program execution. A variable is essentially a name that is assigned to a value. Unlike many other programming languages, Python variables do not require explicit declaration of type. The type of the variable is inferred based on the value assigned.

Variables act as placeholders for data. They allow us to store and reuse values in our program.

Example:

```
# Variable 'x' stores the integer value 10
x = 5

# Variable 'name' stores the string "Samantha"
name = "Samantha"

print(x)
print(name)
```

Output:

```
5
Samantha
```

Rules for Naming Variables

To use variables effectively, we must follow Python's naming rules:

- Variable names can only contain letters, digits and underscores (_).
- A variable name cannot start with a digit.
- Variable names are case-sensitive (myVar and myvar are different).
- Avoid using python keywords (e.g., if, else, for) as variable names.

Valid:

```
age = 21
```

```
_colour = "lilac"
```

```
total_score = 90
```

Invalid:

```
1name = "Error"
```

```
class = 10
```

```
user-name = "Doe"
```

Assigning Values to Variables

Basic Assignment

Variables in Python are assigned values using the=Operator

```
x = 5
```

```
y = 3.14
```

```
z = "Hi"
```

Dynamic Typing

Python variables are dynamically typed, meaning the same variable can hold different types of values during execution.

Examples: x = 10
 x = "Now a string"

Multiple Assignments

Python allows multiple variables to be assigned values in a single line.

Assigning the Same Value

Python allows assigning the same value to multiple variables in a single line, which can be useful for initializing variables with the same value.

Example: `a = b = c = 100`

```
print(a, b, c)
```

Output: 100 100 100

Assigning Different Values

We can assign different values to multiple variables simultaneously, making the code concise and easier to read.

Example: `x, y, z = 1, 2.5, "Python"`

```
print(x, y, z)
```

Output

1 2.5 Python

Python Data Types

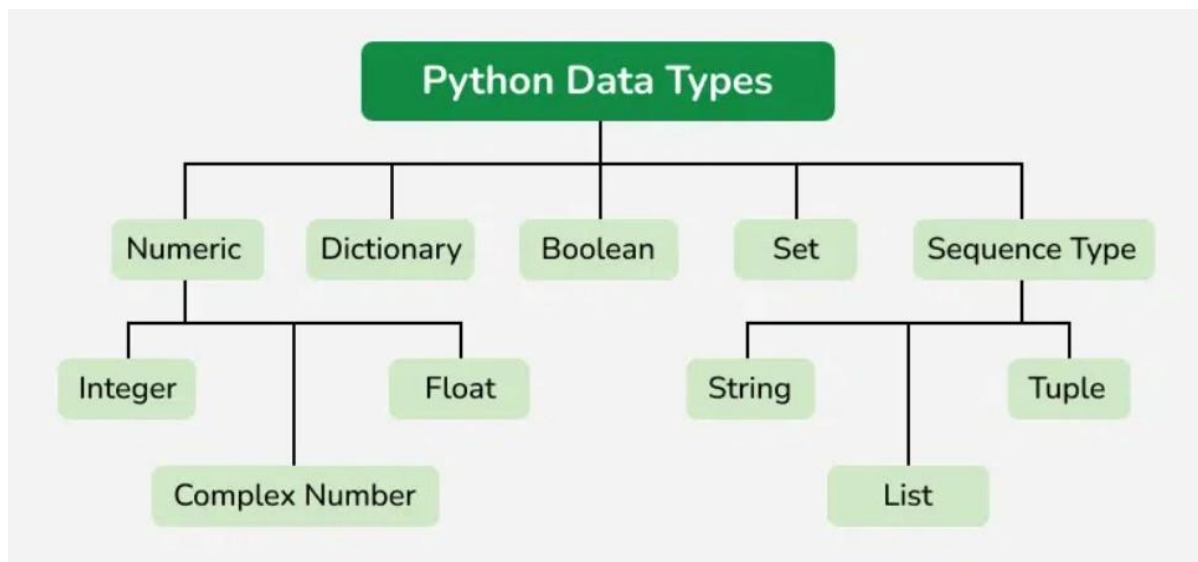


Figure 1.6: Python Data Types

1. Numeric Data Types

Python Numbers represent data that has a numeric value. A numeric value can be an integer, a floating number or even a complex number. These values are defined as int, float and complex classes.

- **Integers:** value is represented by int class. It contains positive or negative whole numbers (without fractions or decimals). There is no limit to how long an integer value can be.
- **Float:** value is represented by float class. It is a real number with a floating-point representation. It is specified by a decimal point. Optionally, character e or E followed by a positive or negative integer may be appended to specify scientific notation.

- **Complex Numbers:** It is represented by a complex class. It is specified as (real part) + (imaginary part)*j*. For example - 2+3*j*

Examples:

```
a = 5
print(type(a))
b = 5.0
print(type(b))
c = 2 + 4j
print(type(c))
```

Output:

```
<class 'int'>
<class 'float'>
<class 'complex'>
```

2. Sequence Data Types

A sequence is an ordered collection of items, which can be of similar or different data types. Sequences allow storing of multiple values in an organized and efficient fashion. There are several sequence data types of Python:

String Data Type

Python Strings are arrays of bytes representing Unicode characters. In Python, there is no character data type, a character is a string of length one. It is represented by str class.

Strings in Python can be created using single quotes, double quotes or even triple quotes. We can access individual characters of a String using index.

Example: s = 'Welcome to the Geeks World'
 print(s)

```
# check data type
print(type(s))

# access string with index
print(s[1])
print(s[2])
print(s[-1])
```

OUTPUT: Welcome to the Geeks World
 <class 'str'>
 e
 l
 d

List

- A **list** is an ordered collection of items.
- It is **mutable** → we can change, add, or remove items after creating it.
- Lists are written using **square brackets []**.

Example: # Empty list

```
a = []
```

```
# list with int values
```

```
a = [1, 2, 3]
```

```
print(a)
```

```
# list with mixed values int and String
```

```
b = ["Geeks", "For", "Geeks", 4, 5]
```

```
print(b)
```

Output: [1, 2, 3]
 ['Geeks', 'For', 'Geeks', 4, 5]

Tuple

- A **tuple** is also an ordered collection of items.
- It is **immutable** → once created, items cannot be changed.
- Tuples are written using **parentheses ()**.

Examples: colors = ("red", "green", "blue")
 print(colors[0])

Output: red

3. Boolean Data Type in Python

Python Boolean Data type is one of the two built-in values, True or False. Boolean objects that are equal to True are *truthy* (true) and those equal to False are *falsy* (false). However non-Boolean objects can be evaluated in a Boolean context as well and determined to be true or false. It is denoted by class bool.

Example: print(type(True))
 print(type(False))
 print(type(true))

Output:
 <class 'bool'>
 <class 'bool'>

Operators and Expression

Python Operators

In Python programming, Operators in general are used to perform operations on values and variables. These are standard symbols used for logical and arithmetic operations.

- **OPERATORS:** These are the special symbols. Eg: - + , * , /, etc.
- **OPERAND:** It is the value on which the operator is applied.

Python has so many built-in operators to perform different arithmetic and logical operations. There are mainly 7 types of operators in Python.

1. Arithmetic Operators
2. Logical Operators
3. Relational Operators
4. Bitwise Operators
5. Assignment operators
6. Identity operators
7. Membership operators

1. Arithmetic operators:

Arithmetic operators are symbols used to perform mathematical operations on numerical values. Arithmetic operators include addition (+), subtraction (-), multiplication (*), division (/), and modulus (%).

Operator	Description	Syntax	Example	Output
+	Addition: adds two operands	$x + y$	$6 + 1$	7
-	Subtraction: subtracts two operands	$x - y$	$5 - 2$	3
*	Multiplication: multiplies two operands	$x * y$	$2 * 3$	6
/	Division (float): divides the first operand by the second	x / y	$10 / 2$	5
//	Division (floor): divides the first operand by the second	$x // y$	$10 // 5$	2 (remainder)

%	Modulus: returns the remainder when the first operand is divided by the second	$x \% y$	$25 \% 5$	0 (remainder)
**	Power: Returns first raised to power second	$x ** y$	$10 ** 5$	100000

Table 1.3: Arithmetic Operators

Python program for arithmetic operations :

```

num1 = 10
num2 = 5
addition = num1 + num2
subtraction = num1 - num2
multiplication = num1 * num2
division = num1 / num2
floor_division = num1 // num2
modulus = num1 % num2
exponentiation = num1 ** num2
print("Addition of num1 and num2 is:", addition )
print("subtraction of num1 and num2 is:", subtraction )
print("multiplication of num1 and num2 is: ", multiplication )
print("division of num1 and num2 is: ", division )
print("floor_division of num1 and num2 is: ", floor_division )
print("modulus of num1 and num2 is: ", modulus)
print("exponentiation of num1 and num2 is: ", exponentiation)

```

Output:

```

Addition of num1 and num2 is: 15
subtraction of num1 and num2 is: 5
multiplication of num1 and num2 is: 50
division of num1 and num2 is: 2.0
floor_division of num1 and num2 is: 2
modulus of num1 and num2 is: 0
exponentiation of num1 and num2 is: 100000

```

2. Logical Operators

Python logical operators are used to combine conditional statements, allowing to perform operations based on multiple conditions. These operators alongside arithmetic operators, are

special symbols used to carry out computations on values and variables. Logical operators are used on conditional statements (either True or False).

They perform **Logical AND**, **Logical OR**, and **Logical NOT** operations.

Operator	Description	Syntax	Example
And	Returns True if both the operands are true	x and y	x>7 and x>10
Or	Returns True if either of the operands is true	x or y	x<7 or x>15
Not	Returns True if the operand is false	not x	not(x>7 and x> 10)

Table 1.4: Logical Operators

Python program for logical operators :

```
a = True
b = False
print(a and b)
print(a or b)
print(not a)
```

output:

```
false
true
false
```

3. Relational Operators

In Python, relational operators are used to compare values.

Operator	Description	Syntax	Example	Return value
<	Less than (Checks if the left value is smaller)	a < b	2 < 6	True
>	Greater than (Checks if the left value is bigger)	a > b	5 > 9	False
<=	Less than or equal to (Returns True, if left is smaller or equal)	a <= b	15 <= 2	False
>=	Greater than or equal to (Returns True, if left is bigger or equal)	a >= b	20 >= 7	False

<code>==</code>	Equal to (Checks if two values are same)	<code>a == b</code>	<code>10 == 10</code>	True
<code>!=</code>	Not equal to (Checks if two values are different)	<code>a != b</code>	<code>6 != 9</code>	True

Table 1.5: Relational Operators

Python program for relational operators :

```

a = 13
b = 33
print(a > b)
print(a < b)
print(a == b)
print(a != b)
print(a >= b)
print(a <= b)

```

Output:

```

False
True
False
True
False
True

```

4. Bitwise Operators:

Python bitwise operators are used to perform bitwise calculations on integers. The integers are first converted into binary and then operations are performed on each bit or corresponding pair of bits, hence the name bitwise operators. The result is then returned in decimal format.

Operator	Description	Syntax	Example	Return value
<code>&</code>	Bitwise AND (Performs bitwise AND on each corresponding bit)	<code>x & y</code>	<code>2 & 6</code>	2
<code> </code>	Bitwise OR (Performs bitwise OR on each corresponding bit)	<code>x y</code>	<code>2 6</code>	6
<code>~</code>	Bitwise NOT (Inverts all the bits)	<code>~x</code>	<code>~6</code>	-7
<code>^</code>	Bitwise XOR	<code>x ^ y</code>	<code>2 ^ 6</code>	4
<code>>></code>	Bitwise right shift	<code>x >> 1</code>	<code>6 >> 1</code>	3

	(Shifts bits to the right, discarding rightmost bits)			
<<	Bitwise left shift (Shifts bits to the left, discarding leftmost bits)	$x \ll 1$	$6 \ll 1$	12

Table 1.6: Bitwise Operators

Truth Table for AND, OR and XOR

A	B	A & B	A B	A ^ B
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Table 1.7: Truth table

Bitwise AND (&):

Binary number of 2 is 0010

Binary number of 6 is 0110

2 => 0010

6=> 0110

2 & 6 => 0010

Result of bitwise XOR operation between 2 & 6 is : 2

Bitwise OR (|) :

Binary number of 2 is 0010

Binary number of 6 is 0110

2 => 0010

6=> 0110

2 | 6 => 0110

Result of bitwise XOR operation between 2 | 6 is : 6

Bitwise XOR (^) :

Binary number of 2 is 0010

Binary number of 6 is 0110

2 => 0010

6=> 0110

2 ^ 6 => 0100

Result of bitwise XOR operation between $2 \wedge 6$ is : 4

Bitwise NOT Operator (~)

- Bitwise NOT is a complement operator.
- It is also called bit flip operator
- It inverts all the bits — changing every 1 to 0 and every 0 to 1.
- It also reverses the sign bit of binary number.

A	~A
1	0
0	1

Table 1.8: NOT Table

How python performs bitwise NOT operator ?

Steps to be followed:

1. Start with the number 6 in binary. $6 = 0110$
2. Invert all bits using (~) $\sim 6 \rightarrow 1001$
3. Interpret the result using two's complement:
 - In two's complement, the first bit (leftmost) indicates the sign (1 = negative, 0 = positive).
 - To find the decimal value of a negative number:
 - Invert the bits back $\rightarrow 0110$
 - Add 1 $\rightarrow 0111$ (decimal 7)
 - Apply the negative sign $\rightarrow -7$
4. Final Result: $\sim 6 = -7$

Another Method for performing bitwise NOT operation using formula :

Formula : $\sim n = -(n + 1)$

Eg : $\sim 6 = -(6+1) = -7$

Bitwise Left Shift Operator (<<)

- This operator shifts the number of bits to the left position.
- It shifts all bits to the left according to the values specified on the right-hand side of the operator.
- After performing the left shift operation on the bits, the operator fills the rightmost bits by 0.
- Mathematically, $x \ll n$ is equivalent to multiplying X by 2^n

Syntax: Operand << n

- **operand:** The value whose bits are to be shifted.
- **n:** The number of positions to shift the bits by.

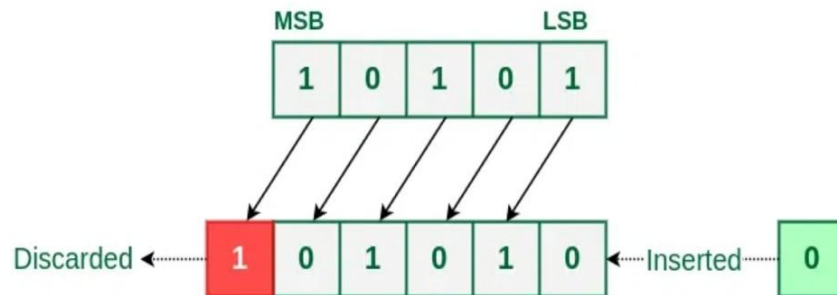


Figure 1.7: Left shift operator

Bitwise Right Shift Operator (>>)

- The bitwise right shift is a binary operator that shifts the number of bits to its right position.
- It shifts all bits to the right according to the values specified on the right-hand side of the operator.
- After performing the right shift operation on the bits, the operator fills the leftmost bits by 0.

Syntax: Operand >> n

- **operand:** The value whose bits are to be shifted.
- **n:** The number of positions to shift the bits by.

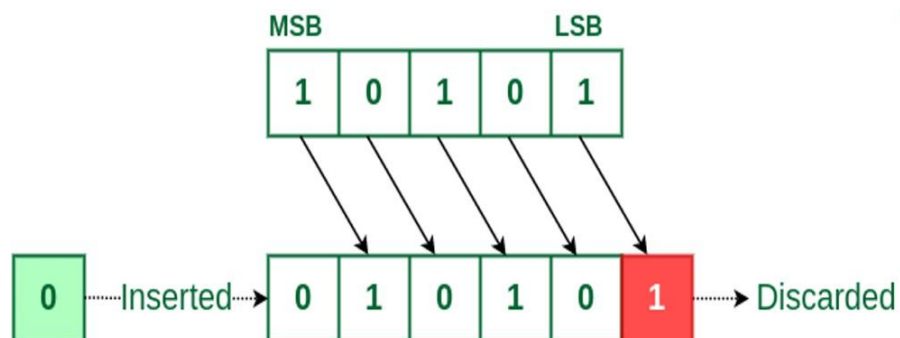


Figure 1.8: Right shift operator

Python program for bitwise operators :

a=1
b=2

```
print(a&b)
print(a|b)
print(a^b)
print(~a)
print(a<<b)
print(a>>b)
```

Output:

```
0
3
3
-2
4
0
```

5. Assignment operators

Assignment Operators are used to assign values to variables. This operator is used to assign the value of the right side of the expression to the left side operand.

Compound assignment operator:

A compound assignment operator is a programming shorthand that performs an operation on a variable and assigns the result back to that same variable in a single step.

Common examples include `+=`, `-=`, `*=`, and `/=`,
where `x += y` is equivalent to `x = x + y`.
where `x -= y` is equivalent to `x = x - y`
where `x *= y` is equivalent to `x = x * y`
where `x /= y` is equivalent to `x = x / y`

These operators provide a more concise way to update a variable's value, especially when the variable is used on both sides of an assignment, making code easier to read and maintain.

Eg: `b += a`

“Take the current value of b, add a to it, and store the result back in b.”

Python program for assignment operators :

```
a = 10
b = a
print(b)
b += a
print(b)
b -= a
```

```
print(b)
b *= a
print(b)
b <<= a
print(b)
```

Output:

```
10
20
10
100
102400
```

6. Identity Operators

In Python, **is** and **is not** are the identity operators both are used to check if two values are located on the same part of the memory. Two variables that are equal do not imply that they are identical.

Operator	Description
Is	It returns true if two variables point the same object and false otherwise
is not	It returns false if two variables point the same object and false otherwise

Table 1.9: Identity Operators

Python program for identity operators :

```
a = 10
b = 20
c = a
print(a is not b)
print(a is c)
```

Output:

```
True
True
```

7. Membership Operators

In Python, **in** and **not in** are the membership operators that are used to test whether a value or variable is in a sequence.

Operator	Description
in	It returns true if value is found in the sequence otherwise it returns False.
not in	It returns true if value is not found in the sequence otherwise it returns False

Table 1.10: Membership Operators

Python program for membership operators :

Eg:1

```
list1 = [1, 2, 3, 4, 5]
print(2 in list1)
print(4 not in list1)
```

output:

True
False

Eg:2

```
message = "Hello World"
print('H' in message)
print('o W' in message)
print('xyz' in message)
print('xyz' not in message)
```

Output:

True
True
False
True

Eg:3

In a dictionary {key : value } ‘ in ’ operator only checks the key not the values.

```
my_dict = {'apple': 1, 'banana': 2}
print('apple' in my_dict)
print(1 in my_dict)
```

Output:

True
False

Expressions in Python

An expression is a combination of operators and operands that is interpreted to produce some other value. In any programming language, an expression is evaluated as per the precedence of its operators. So that if there is more than one operator in an expression, their precedence decides which operation will be performed first.

1. Constant Expressions:

These are the expressions that have constant values only.

Python Code:

```
x = 15 + 1.3  
print(x)
```

Output:

16.3

2. Arithmetic Expressions:

An arithmetic expression is a combination of numeric values, operators, and sometimes parenthesis. The result of this type of expression is also a numeric value. The operators used in these expressions are arithmetic operators like addition, subtraction, etc. Here are some arithmetic operators in Python:

Operators	Syntax	Functioning
+	x + y	Addition
-	x - y	Subtraction
*	x * y	Multiplication
/	x / y	Division
//	x // y	Quotient
%	x % y	Remainder
**	x ** y	Exponentiation

Table 1.11: Arithmetic Expressions

Python Code:

```
x = 40
y = 12
add = x + y
sub = x - y
pro = x * y
div = x / y
print(add)
print(sub)
print(pro)
print(div)
```

Output:

```
52
28
480
3.3333333333333335
```

3. Integral Expressions:

These are the kind of expressions that produce only integer results after all computations and type conversions.

Python Code:

```
a = 13
b = 12.0

c = a + int(b)
print(c)
```

Output:

```
25
```

4. Floating Expressions:

These are the kind of expressions which produce floating point numbers as result after all computations and type conversions.

Python Code:

```
a = 13
b = 5

c = a / b
print(c)
```

Output:

```
2.6
```

5. Relational Expressions:

In these types of expressions, arithmetic expressions are written on both sides of relational operator ($>$, $<$, $>=$, $<=$). Those arithmetic expressions are evaluated first, and then compared as per relational operator and produce a boolean output in the end.

These expressions are also called Boolean expressions.

Python Code:

```
a = 21
b = 13
c = 40
d = 37
p = (a + b) >= (c - d)
print(p)
```

Output:

True

6. Logical Expressions:

These are kinds of expressions that result in either *True* or *False*. It basically specifies one or more conditions.

For example, $(10 == 9)$ is a condition if 10 is equal to 9, it is not correct, so it will return False.

Python code:

```
P = (10 == 9)
Q = (7 > 5)
```

```
R = P and Q
```

```
S = P or Q
```

```
T = not P
```

```
print(R)
```

```
print(S)
```

```
print(T)
```

Output:

False

True

True

7. Bitwise Expressions:

These are the kind of expressions in which computations are performed at bit level.

Python code:

```
a = 12
```

```
x = a >> 2  
y = a << 1
```

```
print(x, y)
```

Output:

3 24

8. Combinational Expressions:

In a combinational expression, different types of expressions are used in a single expression.

Python code:

```
a = 16
```

```
b = 12
```

```
c = a + (b >> 1)
```

```
print(c)
```

Output:

22

Order of Evaluation in Python

The order of evaluation in Python expressions is determined by operator precedence and associativity, primarily following the standard mathematical order of operations.

BODMAS Rule - Order of Operations

Rules of BODMAS in Order	Operations Rules	Examples
B - Brackets	Evaluate expressions within brackets first.	Example: $23 + (5 - 3) - 16/2 + 4 \times 3 + 1$ First solve $(5 - 3)$
O - Orders	Evaluate expressions with exponents or roots.	Example: $23 + 2 - 16/2 + 4 \times 3 + 1$ Then solve (23)
D - Division	Perform division from left to right.	Example: $8 + 2 - 16/2 + 4 \times 3 + 1$ Then solve $(16/2)$
M - Multiplication	Perform multiplication from left to right.	Example: $8 + 2 - 8 + 4 \times 3 + 1$ Then solve (4×3)
A – Addition	Perform addition from left to right.	Example: $8 + 2 - 8 + 12 + 1$ Then solve $8 + 2 + 12 + 1$

S – Subtraction	Perform subtraction from left to right.	Example: 23 - 8 At last, solve 23 - 8 = 15
------------------------	---	--

Table 1.12: BODMAS Rule

Multiple operators in expression (Operator Precedence)

It's a simple process to get the result of an expression if there is only one operator in an expression. But if there is more than one operator in an expression, it may give different results on basis of the order of operators executed. To sort out these confusions, the operator precedence is defined.

Operator Precedence simply defines the priority of operators that which operator is to be executed first.

If we have more than one operator in an expression, it is evaluated as per operator precedence.

Precedence	Name	Operator
1	Parenthesis	() [] {}
2	Exponentiation	**
3	Unary plus or minus, complement	-a , +a , ~a
4	Multiply, Divide, Modulo	/ * // %
5	Addition & Subtraction	+ -
6	Shift Operators	>> <<
7	Bitwise AND	&
8	Bitwise XOR	^
9	Bitwise OR	
10	Comparison Operators	>= <= > <
11	Equality Operators	== !=
12	Assignment Operators	= += -= /= *=
13	Identity and membership operators	is, is not, in, not in

14	Logical Operators	and, or, not
----	-------------------	--------------

Table 1.13: Operator Precedence

Python Code:

```
a = 10 + 3 * 4
print(a)

b = (10 + 3) * 4
print(b)

c = 10 + (3 * 4)
print(c)
```

Output:

```
22
52
22
```

Type Conversion

Type conversion in Python refers to the process of changing the data type. For instance, changing int to str.

There are two types of type conversion in Python:

- **Implicit Conversion** - automatic type conversion
- **Explicit Conversion** - manual type conversion

Implicit Type Conversion

In **implicit conversion**, Python automatically converts one data type into another during expression evaluation. This usually happens when a smaller data type like int is combined with a larger one like float in an operation.

Python Code:

```
x = 10      # Integer
y = 10.6    # Float
z = x + y

print("x:", type(x))
print("y:", type(y))
print("z =", z)
print("z :", type(z))
```

Output:

```
x: <class 'int'>
y: <class 'float'>
z = 20.6
z : <class 'float'>
```

In the above program, **x** is an integer and **y** is a float. In **z = x + y**, Python automatically converts **x** to float to avoid data loss, making **z** a float 0.6.

Note: We get `TypeError`, if we try to add `str` and `int`. For example, `'12' + 23`. Python is not able to use Implicit Conversion in such conditions. So Explicit Type Conversion should be done.

Explicit Type Conversion

This type of conversion is also called typecasting because the user casts (changes) the data type of the objects.

In Explicit Type Conversion, users convert the data type of an object to required data type. We use the built-in functions like `int()`, `float()`, `str()`, etc to perform explicit type conversion.

Python Code:

```
num_string = '12'
num_integer = 23
print(type(num_string))
print(type(num_integer))
# explicit type conversion
num_string = int(num_string)
print(type(num_string))
num_sum = num_integer + num_string
print(type(num_sum))
print(num_sum)
```

Output:

```
<class 'str'>
<class 'int'>
<class 'int'>
<class 'int'>
35
```

In the above example, we have created two variables: `num_string` and `num_integer` with `str` and `int` type values respectively. Notice the code,

```
num_string = int(num_string)
```

Here, we have used `int()` to perform explicit type conversion of `num_string` to integer type.

After converting `num_string` to an integer value, Python is able to add these two variables.

Finally, we got the `num_sum` value i.e 35 and data type to be `int`.

Basis	Implicit Type Conversion	Explicit Type Conversion
Definition	Conversion of data type automatically by Python without user involvement.	Conversion of data type manually by the programmer using built-in functions.
Also Called As	Type Casting (Automatic)	Type Casting (Manual)
Performed By	Python Interpreter	Programmer
Functions Used	Not required (done automatically)	int(), float(), str(), list(), etc.
Example	a = 5 b = 2.5 c = a + b # int → float automatically	x = "100" y = int(x) # string → int manually
Output Type	Determined by Python (based on operation)	Determined by programmer
Risk of Data Loss	Very low, Python avoids it automatically	Possible if not converted carefully
Example Output	7.5, type → <class 'float'>	100, type → <class 'int'>

Table 1.14: Difference between Explicit and Implicit Type Conversion

Notes:

1. Type Conversion is the conversion of an object from one data type to another data type.
2. Implicit Type Conversion is automatically performed by the Python interpreter.
3. Python avoids the loss of data in Implicit Type Conversion. Explicit Type Conversion is also called Type Casting, the data types of objects are converted using predefined functions by the user.
4. In Type Casting, loss of data may occur as we enforce the object to a specific data type.

Input Output Statements

Understanding input and output operations is fundamental to Python programming. With the print() function, we can display output in various formats, while the input() function enables interaction with users by gathering input during program execution.

Input Statements

The input() function is used to take input from the user.
By default, it reads the input as a string.

Syntax: variable = input(prompt) prompt: A string that is displayed on the screen to ask for user input (optional).

1. Basic Input:

```
name = input("Enter your name: ")  
print("Hello, " + name + "!")
```

Output:

Enter your name: apeksha

Hello, apeksha!

Here, the program asks the user for their name and then prints a greeting.

2. Integer Input:

```
age = int(input("Enter your age: "))  
print("You are", age, "years old.")
```

Output:

Enter your age: 28

You are 28 years old.

The input is converted to an integer using `int()` before storing it in the `age` variable.

3. Float Input:

```
Radius=float(input("Enter the radius:"))  
Print("The radius of the circle is:",Radius)
```

Output:

Enter the radius: 3.7

The radius of the circle is 3.7

The input is converted to a float using `float()` before storing it in the `Radius` variable.

Output Statements

The `print()` function is used to display output to the screen.

1. Basic Output:

```
print("Hello, World!")
```

Output:

Hello, World

This prints the string Hello, World!.

2. Printing Multiple Values:

```
name = "Alice"  
age = 25  
print(name, "is", age, "years old.")
```

Output:

Alice is 25 years old.

This prints Alice is 25 years old. with spaces automatically inserted between the arguments.

3. Custom Separator:

```
print("apple", "banana", "cherry", sep=", ")
```

Output:

apple, banana, cherry

This prints apple, banana, cherry.

4. Custom End:

```
print("Hello", end=" ")  
print("World!")
```

Output:

Hello World!

This prints Hello World! on the same line.

5. Printing Without a Newline:

```
for i in range(5):  
    print(i, end=" ")
```

Output:

0 1 2 3 4

This prints 0 1 2 3 4 on the same line.

6. Using format() Method:

```
name = "Bob"  
age = 30  
print("{} is {} years old.".format(name, age))
```

This prints Bob is 30 years old..

7. Using f-strings:

```
name = "Bob"  
age = 30  
print(f'{name} is {age} years old..')
```

This also prints Bob is 30 years old..

Handling Input and Output Together

You can combine input and output to create interactive programs.

Python Code:

```
name = input("What is your name? ")  
print(f'Hello, {name}!')  
age = int(input("How old are you? "))  
print(f'You will be {age + 1} next year..')
```

Output:

What is your name? Dave
Hello, Dave!
How old are you? 32
You will be 33 next year.

This program asks for the user's name and age, then greets them and tells them how old they will be next year.

COMMENTS IN PYTHON

Comments in Python are the lines in the code that are ignored by the interpreter during the execution of the program

- Comments enhance the readability of the code.
- Comment can be used to identify functionality or structure the code-base.
- Comment can help understanding unusual or tricky scenarios handled by the code to prevent accidental removal or changes.
- Comments can be used to prevent executing any specific part of your code, while making changes or testing.

Single-Line Comments

Single-line comments start with the `#` symbol. Anything following the `#` on that line is treated as a comment.

Python Code:

```
# This is a single-line comment  
print("Hello, World!") # This prints a message
```

Output:

Hello,World!

Multi-Line Comments

Python does not have a specific syntax for multi-line comments, but you can create them by using multiple single-line comments or by using triple-quoted strings (`'''` or `"""`). While triple-quoted strings are technically string literals, they can be used as comments if they are not assigned to a variable or used in any other way.

Python Code:

```
""" This is a multi-line comment.
```

It spans multiple lines. """

```
print("Hello, World!")
```

Output:

Hello, World!

Best Practices for Comments

- **Keep Comments Relevant:** Comments should explain why something is done, not what is being done. The code should be self-explanatory whenever possible.
- **Avoid Redundant Comments:** Comments that restate the obvious should be avoided.
- **Update Comments:** Ensure that comments are updated if the related code changes.
